LLD Notes 03 — Design Patterns, Builder, Strategy, and the Trouble with Null

> **Theme:** Design patterns are communication tools, not design tools. Your job is not "use patterns" — your job is readable, maintainable, extendable code. Patterns are names for solutions you'd eventually discover anyway.

---

1. What Design Patterns Are (and Aren't)

In 1994, four authors (the "Gang of Four") catalogued **23 recurring solutions** to recurring object-oriented design problems and gave each one a name: Builder, Strategy, Observer, Factory, Adapter...

The catalogue's real gift was **vocabulary**. Compare:

> "Let's make an object that wraps the payment client and translates its interface into the one our service expects, so the service never imports the vendor SDK."

versus:

> "Let's put an **Adapter** in front of Stripe."

One sentence, full shared understanding. That's what patterns are for — **compressing a design conversation between engineers who share the vocabulary.**

What patterns are NOT

- ❌ A checklist. Nobody good goes to work thinking "which design pattern shall I use today?" Code written that way is over-engineered and reads like a patterns textbook exploded.
- ❌ A measure of seniority. Cramming five patterns into a feature is a junior move wearing a senior costume.
- ❌ Universal. Many patterns exist only to **patch a gap in a specific language.** When the language grows the feature, the pattern dissolves (you'll see this happen *twice* in these notes).

The right mental model

1. Focus on the fundamentals: **readable, maintainable, extendable** code — managing dependencies, one responsibility per class, behavior next to data.
2. Good design naturally converges on pattern-shaped solutions. Then you *recognize* the pattern and gain its name for free.
3. Use the name to communicate, in code review and in design docs — not to decide.

> Patterns should be **discovered in your code, not installed into it.**

---

2. Builder Pattern

2.1 The problem: telescoping constructors

You're modeling a coffee order. Five optional aspects: size, shots, milk, sugar, decaf.

java

```java
public class Coffee {
    private final String size;
    private final int shots;
    private final boolean milk;
    private final int sugarCubes;
    private final boolean decaf;

    // which constructors do we provide?
    Coffee(String size) { ... }
    Coffee(String size, int shots) { ... }
    Coffee(String size, int shots, boolean milk) { ... }
    Coffee(String size, boolean milk, int sugarCubes) { ... }     // wait, this
    Coffee(String size, int shots, boolean milk, int sugarCubes, boolean decaf)
    ...
}
```

Count the combinations of "which parameters does the caller want to supply":

$$ {}^{n}C_0 + {}^{n}C_1 + {}^{n}C_2 + \ldots + {}^{n}C_n = 2^n $$

5 optional fields → $2^5 = 32$ possible constructors
10 fields → 1,024

You obviously can't write 2^n constructors — many wouldn't even compile (same parameter types, different meanings). So people write the **telescoping** compromise — each constructor forwarding to a bigger one — and callers end up with this:

java

```java
// what does this even mean? true what? 0 what?
var coffee = new Coffee("LARGE", 2, true, 0, false);
```

Unreadable at the call site, fragile to reorder, and every new field multiplies the pain.

2.2 The solution: a fluent Builder

Move construction into a mutable helper with **named steps**, then produce the immutable object once, at `build()`:

java

```java
public record Coffee(
        String size,
        int shots,
        boolean milk,
        int sugarCubes,
        boolean decaf
) {
    // entry point: Coffee.builder()
    public static Builder builder() {
        return new Builder();
    }

    public static class Builder {
        // defaults live in ONE place
        private String size = "MEDIUM";
        private int shots = 1;
        private boolean milk = false;
        private int sugarCubes = 0;
        private boolean decaf = false;

        public Builder size(String size)      { this.size = size; return this;
        public Builder shots(int shots)       { this.shots = shots; return this
        public Builder milk()                 { this.milk = true; return this;
        public Builder sugar(int cubes)       { this.sugarCubes = cubes; return
        public Builder decaf()                { this.decaf = true; return this;

        public Coffee build() {
            if (shots < 1 || shots > 5)
                throw new IllegalArgumentException("1..5 shots");   // validate
            return new Coffee(size, shots, milk, sugarCubes, decaf);
        }
    }
}
```

The call site now reads like the order itself:

java

```java
var coffee = Coffee.builder()
        .size("LARGE")
        .shots(2)
        .milk()
        .build();
```

What we gained:

Telescoping constructors	Builder
`new Coffee("LARGE", 2, true, 0, false)` — positional soup	every value labeled at the call site
2^n combinations to cover	one builder covers all combinations
defaults duplicated across constructors	defaults in one place
easy to swap two `int` args silently	impossible — steps are named
object may be seen half-configured	object is born complete and **immutable**

Where you already use it: `StringBuilder`, `Stream.builder()`, `HttpRequest.newBuilder()...build()` in the JDK; `@Builder` from Lombok generates exactly the class above.

2.3 The twist: Builder is a patch over a language gap

Python has **named parameters with defaults.** The entire problem evaporates:

python

```python
@dataclass(frozen=True)
class Coffee:
    size: str = "MEDIUM"
    shots: int = 1
    milk: bool = False
    sugar_cubes: int = 0
    decaf: bool = False

coffee = Coffee(size="LARGE", shots=2, milk=True)   # ...that's it. That's the
```

No builder class, no fluent chaining — because the *language* names the arguments. Kotlin, Swift, C#, and Scala are the same.

The lesson generalizes: a large share of the classic 23 patterns are workarounds for missing language features, and they fade as languages evolve. Builder fades under named parameters. Iterator faded into `for-each`. Singleton (mostly) faded into enums and DI containers. And in the next section, Strategy fades into a lambda.

So when someone asks *"why don't Python developers talk about Builder?"* — the honest answer is: **their language already shipped it.**

---

3. Strategy Pattern

3.1 The problem, as your product manager creates it

Day 1: "Write me a function that sums a list of integers."

```java
public static int sum(List<Integer> nums) {
    int s = 0;
    for (int x : nums) s += x;
    return s;
}
```

Day 7: "Actually, I need the sum of only the **even** numbers."

```java
public static int sumEven(List<Integer> nums) {
    int s = 0;
    for (int x : nums)
        if (x % 2 == 0) s += x;
    return s;
}
```

Day 9: "And one for **odd**." **Day 12:** "Only positives." **Day 15:** "Only multiples of 5, we're doing a promotion."

Look at what you're doing: copying the same loop and changing **one line** — the decision of *which numbers to include*. The loop is stable; the decision is volatile. **Separate what changes from what stays the same.**

3.2 The insight: let the CALLER supply the decision

Stop deciding inside the function. Take the decision as a **parameter**:

```java
public static int sum(List<Integer> nums, Predicate<Integer> include) {
    int s = 0;
    for (int x : nums)
        if (include.test(x)) s += x;
    return s;
}
```

Now every future requirement is a **call**, not a new function:

```java
```

```
var nums = List.of(3, 1, 2, 4, 5, 7, 9, 8);

sum(nums, x -> true);            // everything      → 39
sum(nums, x -> x % 2 == 0);      // evens           → 14
sum(nums, x -> x % 2 != 0);      // odds            → 25
sum(nums, x -> x > 4);           // PM's next idea  → no code change on our sid
```

This is the **Strategy pattern**: *encapsulate a family of interchangeable algorithms behind one interface, and let the client pick which one to inject.* The `sum` function is **closed for modification, open for extension** — OCP, achieved by one parameter.

3.3 The heavyweight version (how we did it before lambdas)

Before Java 8, "pass behavior as a value" required a ceremony of types:

java

```
// 1. an interface for the strategy
interface InclusionStrategy {
    boolean include(int x);
}

// 2. one CLASS per behavior
class IncludeAll implements InclusionStrategy {
    public boolean include(int x) { return true; }
}
class IncludeEven implements InclusionStrategy {
    public boolean include(int x) { return x % 2 == 0; }
}
class IncludeOdd implements InclusionStrategy {
    public boolean include(int x) { return x % 2 != 0; }
}

// 3. the context accepts the interface
static int sum(List<Integer> nums, InclusionStrategy strategy) {
    int s = 0;
    for (int x : nums)
        if (strategy.include(x)) s += x;
    return s;
}

// 4. the caller instantiates a whole object to say "even numbers"
sum(nums, new IncludeEven());
```

Four files to express `x % 2 == 0`. It works — this *is* Strategy, and millions of lines of pre-2014 Java look exactly like this — but the ceremony buries the idea.

3.4 The lightweight version: a lambda IS a strategy

Java 8 noticed that an interface with one abstract method is really just *a function that needed a name*. It gave us `@FunctionalInterface`, lambdas, and method references — and the pattern collapsed:

```java
sum(nums, x -> x % 2 == 0);          // the strategy, inline
sum(nums, Main::isEven);             // or a method reference to existing logic

static boolean isEven(int x) {
    return x % 2 == 0;               // note: the whole if/return-true/return-f
}                                    //       is just... the expression itself
```

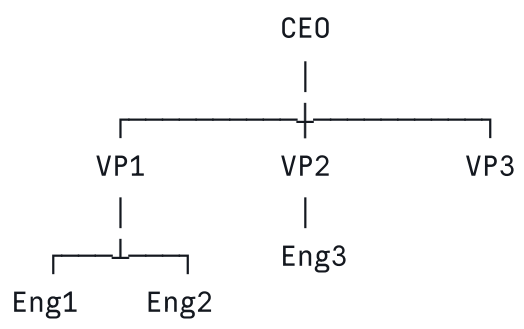
And the modern loop body itself dissolves into streams:

```java
public static int sum(List<Integer> nums, Predicate<Integer> include) {
    return nums.stream()
               .filter(include)
               .mapToInt(Integer::intValue)
               .sum();
}
```

Same story as Builder: **the language absorbed the pattern.** The heavyweight interface-plus-classes version still matters when a strategy has state, multiple methods, or needs its own dependencies — but for "one decision as a value," a lambda *is* the Strategy pattern, minus the scaffolding. You've been using Strategy every time you call `.filter(...)`, `.sorted(comparator)`, or `.map(...)`.

3.5 A real problem: appraisals across an org tree

Now a strategy worth the name. HR asks: *give every manager an appraisal based on the salaries of **all** their subordinates — direct and indirect.*



The **traversal** of this tree is stable. The **formula** — mean of subordinates? median? something creative? — is exactly the part HR will keep changing. Sound familiar? Same

shape as `sum`: stable skeleton, volatile decision. Inject the decision.

java

```java
// the org tree — an immutable record, naturally recursive
public record Employee(String name, int salary, List<Employee> reports) {

    public Employee {
        reports = List.copyOf(reports);
    }

    /** every salary below this node — direct AND indirect reports */
    public List<Integer> allSubordinateSalaries() {
        return reports.stream()
                .flatMap(r -> Stream.concat(
                        Stream.of(r.salary()),
                        r.allSubordinateSalaries().stream()))
                .toList();
    }
}
```

java

```java
// the stable skeleton: walk the tree, apply SOME formula, rebuild (records are
public static Employee applyAppraisal(Employee e,
                                      Function<List<Integer>, Integer> appraisa
    var updatedReports = e.reports().stream()
            .map(r -> applyAppraisal(r, appraisalStrategy))     // recurse firs
            .toList();

    var subSalaries = e.allSubordinateSalaries();
    int raise = subSalaries.isEmpty() ? 0 : appraisalStrategy.apply(subSalaries

    return new Employee(e.name(), e.salary() + raise, updatedReports);
}
```

java

```
// the volatile part: three formulas HR proposed this quarter — each is just a
static int meanStrategy(List<Integer> salaries) {
    return (int) salaries.stream().mapToInt(Integer::intValue).average().orElse
}

static int medianStrategy(List<Integer> salaries) {
    var sorted = salaries.stream().sorted().toList();
    int n = sorted.size();
    return (n % 2 == 1) ? sorted.get(n / 2)
                        : (sorted.get(n / 2 - 1) + sorted.get(n / 2)) / 2;
}

static int minMaxStrategy(List<Integer> salaries) {
    var stats = salaries.stream().mapToInt(Integer::intValue).summaryStatistics
    return 2 * stats.getMax() - stats.getMin();        // HR's "creative" formu
}
```

java

```
// swapping the entire company's appraisal policy = swapping ONE argument
var afterMean   = applyAppraisal(ceo, Main::meanStrategy);
var afterMedian = applyAppraisal(ceo, Main::medianStrategy);
var afterMinMax = applyAppraisal(ceo, Main::minMaxStrategy);
```

Notice what did **not** happen: no re-writing of the traversal, no `switch(policyType)` inside it, no `AppraisalStrategyFactoryImpl`. The tree-walk was written once; policies are plug-in functions. When HR invents formula #4 next quarter, it's five lines *outside* the traversal.

> **Strategy in one sentence:** find the line that keeps changing, and turn that line into a parameter.

---

4. Null — "The Billion-Dollar Mistake"

Tony Hoare invented the null reference in 1965 for ALGOL W. His own retrospective:

> "I call it my billion-dollar mistake... it has probably caused a billion dollars of pain and damage in the last forty years."

4.1 Why null is a smell

It explodes far from the crime scene. The method that *returned* null is innocent-looking; the crash happens later, somewhere else:

java

```
String getName() {
    return name;                          // quietly null
}

getName().toUpperCase();                  // 💥 NullPointerException — HERE, not t
```

It's invisible in the type. The signature says `String`. It's lying — it actually means "a String, *or a landmine*." The compiler can't warn you because the type system has no way to say "this might be absent."

Defending against it poisons everything. Once null is possible, every caller everywhere must check — or gamble:

java

```
if (getName() != null) { ... }                          // check-pollution, everyw
if (a != null && a.getB() != null && a.getB().getC() != null) { ... }   // chai
```

4.2 Java's answer: `Optional<T>` — absence as a type

`Optional<T>` makes "might be absent" **visible in the signature** and forces the caller to confront it:

java

```
Optional<Question> findById(long id);        // the signature now TELLS THE TRU
```

java

```
// the caller can no longer "forget" — the type demands a decision:

// provide a fallback
String title = repo.findById(42)
        .map(Question::title)
        .orElse("(deleted question)");

// act only if present
repo.findById(42).ifPresent(q -> System.out.println(q.title()));

// or convert absence into a domain error — our service does exactly this
Question q = repo.findById(id)
        .orElseThrow(() -> new NoSuchElementException("question " + id + " not
```

You've already been using this: our `QuestionRepository.findById` returns `Optional<Question>`, and the service's `orElseThrow` is what becomes the HTTP 404.

4.3 But you cannot stop a bad programmer with a type

`Optional` is a convention, not a guarantee. This compiles:

```
java

Optional<Question> findById(long id) {
    return null;                          // 💀 congratulations, you've made it WO
}                                         //    now callers trust the signature an
```

So does `optional.get()` with no check — an NPE wearing a nicer name. **No language feature survives determined negligence.** Which is why teams adopt *ground rules* rather than relying on the type alone:

Ground rules for null (adopt these as personal law)

1. **Never return `null`. From anything. Ever.**
2. Method might legitimately have no answer → return **`Optional<T>`** (and never a null Optional).
3. Method returns a collection and there's nothing to return → return **empty**, not null: `List.of()`, `Map.of()`. A caller can loop over an empty list safely; null makes them check first.
4. **`Optional` is for return types only** — not fields, not method parameters, not collection elements. (`Optional<List<T>>` and `List<Optional<T>>` are both design smells.)
5. Never call `.get()` bare — use `orElse`, `orElseThrow`, `map`, `ifPresent`.
6. At system boundaries (JSON, DB) where null sneaks in, **normalize immediately** — our records already do this pattern:

```
java

public QuestionFilter {
    tags = (tags == null) ? List.of() : List.copyOf(tags);   // null dies at th
}
```

4.4 Other languages fixed this in the type system

Kotlin (and Swift, C#, TypeScript in strict mode) splits every type in two — `String` **cannot** hold null; `String?` can, and the **compiler** forces the check:

```
kotlin
```

```
fun greet(name: String, nickname: String?) {     // name can NEVER be null; nic
    println(name.uppercase())                    // ✓ no check needed — guarant
    println(nickname.uppercase())                // ✗ DOES NOT COMPILE
    println(nickname?.uppercase() ?: "no nick")  // ✓ compiler-enforced handlin
}
```

The null check moved from *runtime discipline* to *compile-time proof*. Same pattern as Builder and Strategy one more time: **Java's `Optional` is a library patching a language gap that newer languages closed in the type system itself.** (Java is inching there too — `@Nullable`/JSpecify annotations let tools flag violations — but it's tooling, not the language.)

---

5. Summary

1. **Patterns are vocabulary.** They compress design conversations; they don't replace design thinking. Discover them in your code; don't install them.
2. **Builder** solves the 2^n telescoping-constructor explosion with named fluent steps, single-point defaults, validation at `build()`, and an immutable result. In languages with named parameters, the pattern is a built-in.
3. **Strategy** = separate the stable skeleton from the volatile decision, and inject the decision. The heavyweight form is interface + implementation classes; **a lambda is a lightweight strategy** — the language absorbed the pattern. You use it in every `.filter(...)`.
4. The appraisal tree shows Strategy earning its keep: one traversal, unlimited HR formulas, zero rewrites.
5. **Null lies about types and explodes at a distance.** Return `Optional` or empty collections, never null; kill null at your boundaries; and remember that discipline, not the type, is the real defense — newer languages moved that discipline into the compiler.
6. Recurring meta-lesson (three times today): **many design patterns are missing language features.** Judge every pattern by asking: *what gap is this filling — and does my language still have that gap?*

Appendix — The Pattern Catalogue (a map, not a to-do list)

Below is the classic Gang-of-Four catalogue plus a few modern additions, for reference only. **Do not memorize this to "apply" it.** Let the pattern emerge from the problem you're solving — then come here to learn its name so you can communicate it. If you find yourself picking a pattern first and hunting for a place to use it, you're holding the map upside down.

You've already *met* several of these without ceremony: Builder and Strategy (today), Repository, DTO/Transformer, Immutable Object, and Dependency Injection (Stack Overflow API), Composite and Template Method (hiding inside the appraisal tree — a tree

of employees, a fixed traversal with a pluggable step). That's how it should feel every time: the problem first, the name after.

Creational — 5

Pattern	One line
Abstract Factory	Create families of related objects
Builder	Construct complex objects step-by-step
Factory Method	Defer object creation to subclasses
Prototype	Create objects by cloning existing instances
Singleton	Ensure a class has a single instance

Structural — 7

Pattern	One line
Adapter	Make incompatible interfaces work together
Bridge	Separate abstraction from implementation
Composite	Treat individual objects and compositions uniformly
Decorator	Add behavior dynamically
Facade	Provide a simplified interface to a subsystem
Flyweight	Share common state to reduce memory
Proxy	Control access to another object

Behavioral — 11

Pattern	One line
Chain of Responsibility	Pass a request through a chain of handlers
Command	Encapsulate a request as an object
Interpreter	Represent and evaluate a grammar
Iterator	Traverse a collection without exposing internals
Mediator	Centralize communication between objects
Memento	Capture and restore object state
Observer	Notify dependents when state changes
State	Change behavior based on internal state
Strategy	Encapsulate interchangeable algorithms
Template Method	Define algorithm skeleton, defer steps to subclasses
Visitor	Add operations to object structures without changing them

Beyond the 23 — and there are many more

Pattern	One line
Dependency Injection	Supply dependencies from outside instead of constructing them inside
Null Object	Replace null with a do-nothing object so callers never check
Object Pool	Reuse expensive objects instead of creating/destroying them
Repository	Hide data access behind a collection-like interface
Immutable Object	Objects that never change after construction
Composition over Inheritance	Assemble behavior from parts instead of inheriting it
Fluent Interface / Pipeline	Chain calls that read like a sentence (`stream().filter().map()`)
Lazy Evaluation	Compute a value only when (and if) it's needed
Futures / Async	Represent a value that will exist later

The catalogue keeps growing because problems keep recurring — that's all a pattern is: **a recurring problem with a named, agreed-upon solution.** Learn the fundamentals, solve the problem in front of you, and the names will attach themselves.